



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Fecha:

DD/MM/2026

Analisis y Diseño de Sistemas 1

Escuela de Ingeniería de Ciencias Y Sistemas

NOMBRE DEL TUTOR

UNIDAD 3

Casos de uso

Modelado y Construcción de

Sistemas

Anuncios Importantes



Foro Semanal



Anuncio 2



Anuncio 3



Anuncio 4



Anuncio 5

Agenda



Foro Semanal



Asignacion DTT



TEMA 3



TEMA 4



TEMA 5

COMPETENCIA(S) QUE DESARROLLAREMOS



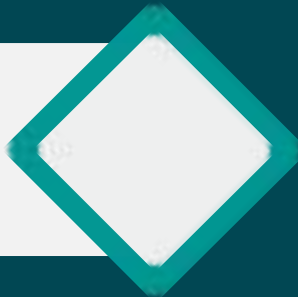
Modelar sistemas de software utilizando Casos de Uso del Negocio (CUN) y el Lenguaje de Modelado Unificado (UML), identificando actores, relaciones y niveles de abstracción, con el fin de especificar requisitos funcionales de manera clara, estructurada y documentada, aplicando herramientas de modelado estándar de la industria.

PACIENCIA

El modelado de sistemas exige detenerse a pensar antes de actuar. Dibujar un diagrama, documentar un caso de uso o identificar una relación incorrecta no es perder el tiempo, es invertirlo. El desarrollador paciente que modela bien desde el inicio entrega sistemas más sólidos, comete menos errores y trabaja con mayor claridad que quien simplemente empieza a codificar sin un plan.

CONCEPTOS CLAVE APRENDIDOS

- Los Casos de Uso del Negocio nos permiten capturar QUÉ debe hacer un sistema desde la perspectiva del actor, utilizando tres niveles: Core (visión general), Descomposición (relaciones entre CUN) y Expandido (flujos detallados). Las relaciones «include», «extend» y generalización definen cómo se conectan entre sí. Todo queda registrado en una plantilla estándar.
- UML complementa este proceso con diagramas que representan la estructura del sistema (clases, atributos, relaciones) y su comportamiento en el tiempo (secuencia de mensajes entre objetos).
- Modelar correctamente antes de construir reduce errores, mejora la comunicación del equipo y produce software de mayor calidad.



CASOS DE USO

Modelando cómo el negocio interactúa con el sistema.



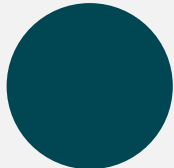
¿QUÉ SON LOS CASOS DE USO?

Un caso de uso es una situación que ocurre dentro del negocio o sistema, protagonizada por actores, que conlleva un conjunto de pasos listados en lenguaje natural. Describe qué hace el sistema desde el punto de vista de quien lo usa — no cómo lo hace internamente.

Dicho de otra forma: un caso de uso captura una necesidad del negocio y la expresa como una secuencia de acciones y reacciones entre el actor y el sistema, de manera que cualquier persona involucrada en el proyecto pueda entenderla, sin importar si es técnica o no.



Características:

- Introducidos por Ivar Jacobson como respuesta a la carencia de métodos para capturar requisitos
 - Describen el comportamiento del sistema desde la perspectiva del usuario
 - Basados en lenguaje natural: accesibles para usuarios no técnicos
 - Son independientes de la implementación (no importa la tecnología)
 - Definen los límites del sistema y sus relaciones con el entorno
 - Particionan el conjunto de necesidades según la categoría de usuarios que participan
- 

CASOS DE USO DEL NEGOCIO vs. DEL SISTEMA

¿Cuál es la diferencia?

CAMPO	Casos de Uso del Negocio (CUN)	Casos de Uso del Sistema (CUS)
Nivel	Proceso de negocio completo	Funcionalidad específica del software
Actor	Actor del negocio (cliente, proveedor, gerente)	Usuario del sistema, sistemas externos
Notación	Óvalo con línea diagonal /	Óvalo simple
Ejemplo	Gestionar reservas de una cafetería	Eliminar Reserva

Primero modelamos el negocio — qué hace la empresa, qué procesos tiene, quiénes participan. Luego, al bajar al detalle del sistema aparecen funcionalidades.

¿En qué momento del proyecto usamos los CU?

MODELADO DEL NEGOCIO

CU DEL NEGOCIO



Se usa el **CORE** y la **PRIMERA DESCOMPOSICIÓN** para entender qué hace la empresa, delimitar el alcance y acordar los procesos con los **STAKEHOLDERS**.



(CORE + PRIMERA DESCOMPOSICIÓN)

CAPTURA DE REQUISITOS

CU DEL SISTEMA



Se expanden esos procesos en **CASOS DE USO SISTEMA**, de donde emergen los **REQUISITOS FUNCIONALES**.



(CU EXPANDIDOS)

ACTORES DEL NEGOCIO

¿quiénes participan en el negocio?

Un actor es un rol, no una persona específica, que interactúa con el negocio.

Tipos de actores candidatos:

- Clientes o clientes potenciales
- Socios y proveedores
- Autoridades externas
- Sistemas de información externos
- Otras áreas de la organización

Reglas importantes:

- Cada actor tiene un nombre y descripción de su rol
- Cada actor se involucra con al menos un caso de uso
- Un actor puede heredar el rol de otro (generalización entre actores)

- Los stakeholders tienen visión parcial del negocio — hay que escuchar a todas las partes
- El ingeniero de requisitos es responsable de capturar los requisitos de forma completa
- No modelar en base a supuestos: si no hay respuesta, escalar al gerente
- Los dueños de los procesos de negocio son los jefes de área, no los usuarios operativos





EL CORE DEL NEGOCIO (acuerdo general)

Paso 1: Identificar el Core del Negocio

El core es el primer y único óvalo que hacemos: representa la totalidad de lo que se va a modelar.

No siempre es todo el negocio de la empresa — es específicamente lo que nos están pidiendo, que muchas veces es solo una pequeña parte del proceso completo de la organización. Es solo una descripción general, pero tiene que estar muy bien delimitada y englobar la totalidad del sistema a modelar.

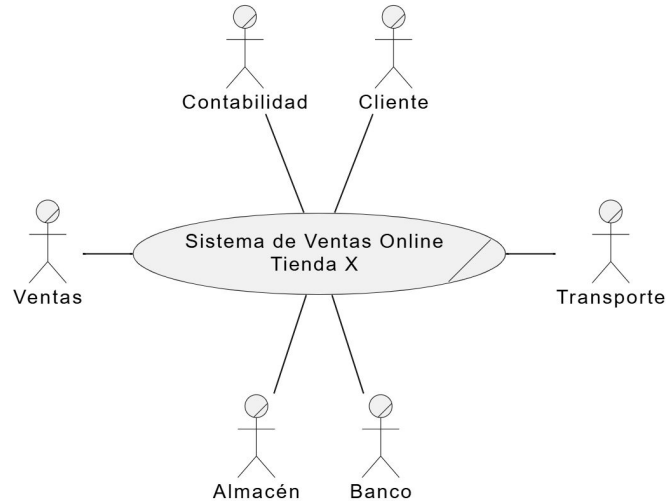


¿Cómo se construye el diagrama del core?

1. Entender el negocio — hablar con stakeholders, estudiar el dominio
2. Pensar primero en los procesos que componen lo que vamos a modelar
3. Identificar los actores más importantes que interactúan con ese negocio
4. Nombrar el core de forma representativa: que todos los stakeholders lo reconozcan al leerlo
5. Validar con los stakeholders — si el nombre y los actores no reflejan el sistema para ellos, hay que ajustar

¿Cómo se ve un CU - Core del Negocio?

Diagrama



Descripción

Sistema que gestiona las ventas en línea de Tienda X, desde la recepción del pedido del cliente hasta su entrega, incluyendo gestión de inventario, pagos y coordinación logística.

PRIMERA DESCOMPOSICIÓN (procesos del negocio)

Paso 2: Descomposición del Core

Una vez que el core está validado, lo descomponemos en sus procesos del negocio. Estos son los casos de uso del negocio también de alto nivel,, los que definen qué hace el sistema en términos generales.

¿Qué es un proceso del negocio?

Un grupo de tareas lógicamente relacionadas que emplean recursos de la organización y producen un resultado de valor observable para los actores. Es un flujo de trabajo completo — no una tarea suelta ni un paso aislado.

PRIMERA DESCOMPOSICIÓN (procesos del negocio)

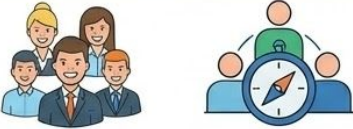
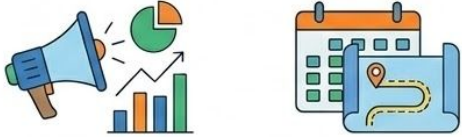
Paso 2: Descomposición del Core

Cada proceso que identificamos en esta etapa se convierte en un óvalo con la línea diagonal en el diagrama. Nosotros no inventamos estos procesos — los descubrimos estudiando cómo funciona el negocio. Si no existen, no podemos modelarlos.

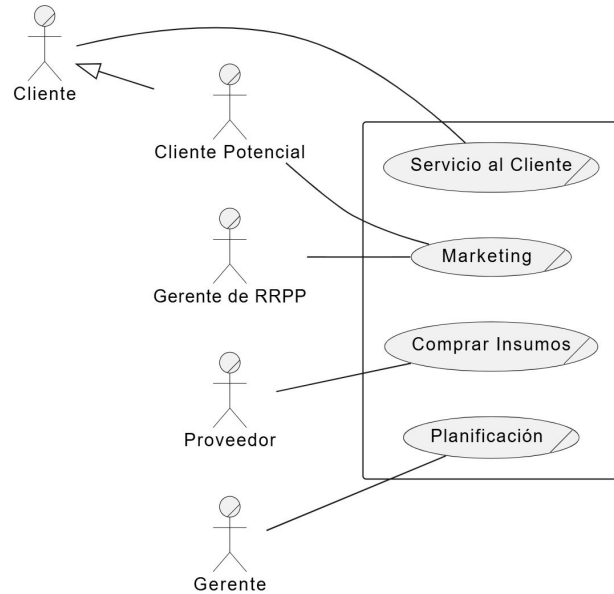
¿Quién es responsable de qué?

- Los jefes de área son los dueños de los procesos: ellos definen cómo funciona el negocio
- Nosotros modelamos esos procesos como casos de uso
- Si un proceso no está definido, podemos apoyar a identificarlo, pero no es nuestra obligación crearlo
- Si los stakeholders no responden, hay que comunicarlo y escalar — no suponer

TIPOS DE PROCESOS A IDENTIFICAR

[TIPO]	[DESCRIPCIÓN]	[EJEMPLO (CAFETERÍA)]
Nucleares 	Esenciales — sin ellos el negocio no opera.	Servicio al cliente, Preparar pedidos 
Gerenciales 	Estrategia, coordinación, relaciones.	Marketing, Planificación 
De Soporte 	Apoyan a los nucleares.	Comprar insumos, Gestión de personal 

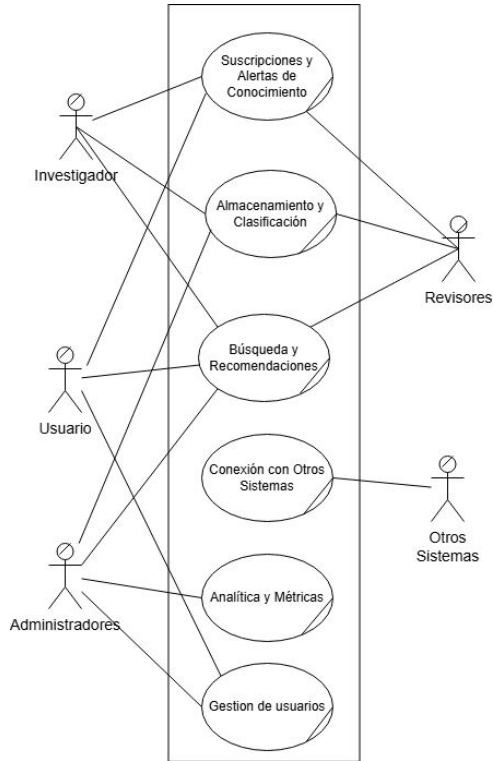
¿Cómo se ve un CU - de Primera descomposición?



Nota sobre notación:

- Línea sin flecha: comunicación bidireccional — el actor lleva y extrae información
- Flecha hacia el actor: el proceso envía información al actor
- Flecha hacia el CU: el actor envía información al proceso
- Línea de generalización (triángulo vacío): el actor hijo hereda el rol del padre

¿Cómo se ve un CU - de Primera descomposición?



Hay que buscar identificar la totalidad de los procesos antes de avanzar a expandir.

Esto sirve para delimitar el alcance con claridad — cada uno de estos procesos se va a expandir en la siguiente etapa para encontrar los requisitos funcionales. Si nos saltamos uno, ese requisito queda sin cubrir.

CASOS DE USO EXPANDIDOS

Paso 3: Expandir los Casos de Uso del Negocio

¿Por qué expandimos?

La primera descomposición nos da los procesos del negocio a nivel general. Pero para encontrar los requisitos funcionales, necesitamos bajar al detalle: ¿quiénes participan exactamente?, ¿qué pasos ocurren?, ¿qué variaciones existen? Eso es expandir.

Al expandir, pueden aparecer

- Nuevos actores que no eran visibles en el nivel anterior
- Pasos opcionales que se modelan como <<extend>>
- Comportamientos repetidos que se modelan como <<include>>
- Variantes de un mismo proceso que se modelan como generalización

Casos de Uso ≠ Diagramas de Flujo

Casos de Uso:

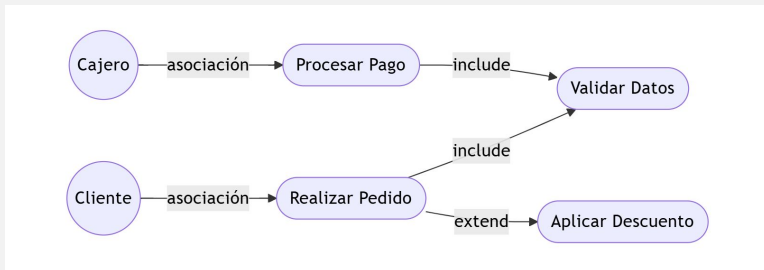
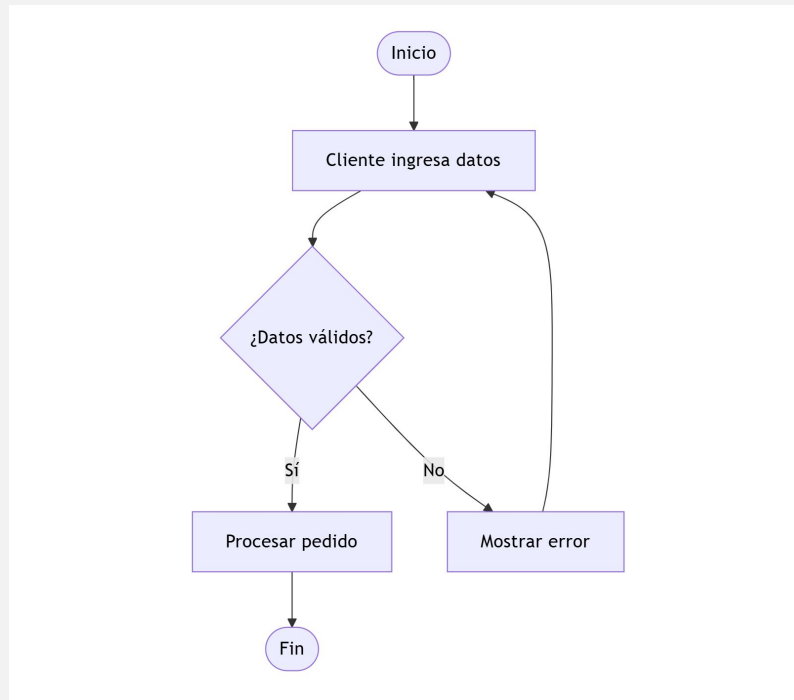


Diagrama de flujo:



LAS TRES RELACIONES EN LOS CU - EXPANDIDOS

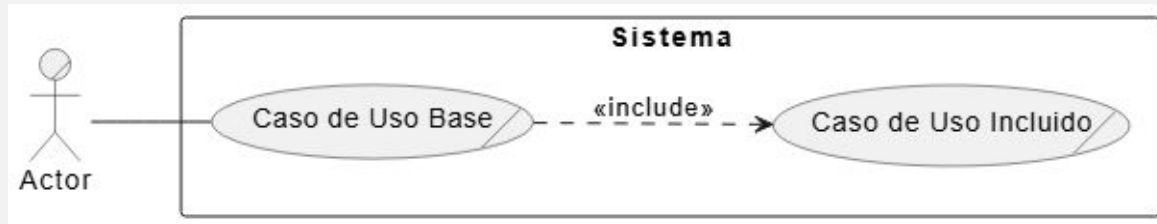
RELACIÓN 1: <<include>>

¿Qué es?

- El caso de uso base siempre incorpora el comportamiento del caso de uso incluido en un punto específico del flujo.

¿Cuándo usarla?

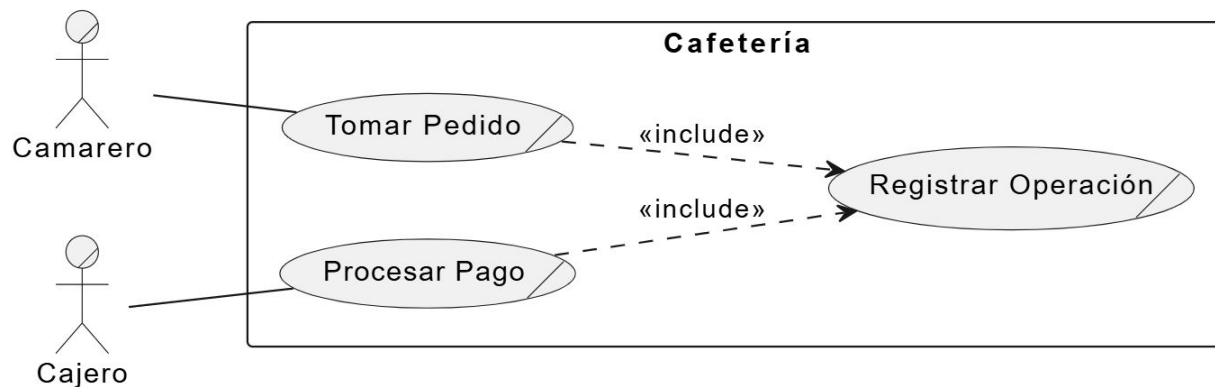
- Cuando un comportamiento se repite en varios casos de uso y conviene no describirlo dos veces
- Cuando el comportamiento incluido siempre ocurre — no es opcional



LAS TRES RELACIONES EN LOS CU - EXPANDIDOS

RELACIÓN 1: <<include>>

Siempre que se toma un pedido, se registra la operación. Siempre que se procesa un pago, también se registra. El "Registrar Operación" no se repite — se incluye desde ambos.



LAS TRES RELACIONES EN LOS CU - EXPANDIDOS

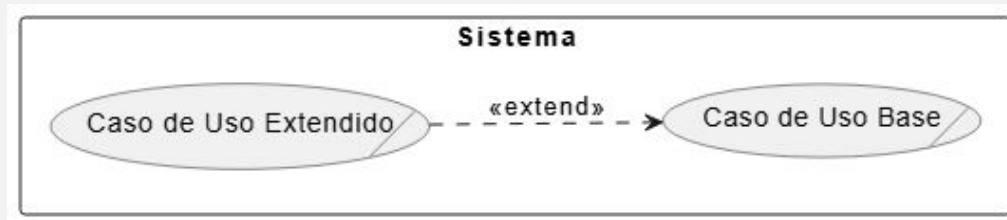
RELACIÓN 2: <<extend>>

¿Qué es?

- El caso de uso extendido puede modificar el comportamiento del base bajo ciertas condiciones — pero el base no sabe cómo ocurre esa extensión. Solo sabe que hay puntos donde puede ser extendido.

¿Cuándo usarla?

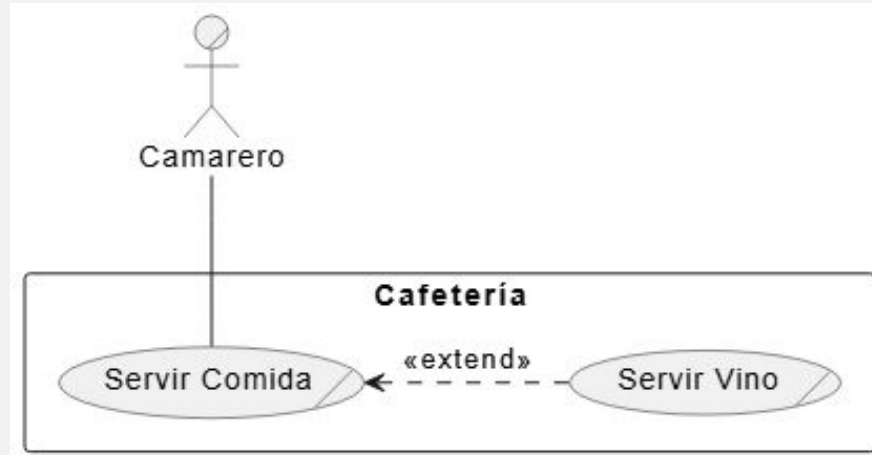
- Comportamiento opcional que ocurre solo bajo ciertas condiciones
- Flujos alternativos que raramente se ejecutan
- El caso de uso base sigue siendo completo por sí solo, con o sin la extensión



LAS TRES RELACIONES EN LOS CU - EXPANDIDOS

RELACIÓN 2: <<extend>>

Servir vino solo ocurre si el cliente lo pidió. La comida se sirve igual aunque no haya vino — el base es completo por sí solo.



LAS TRES RELACIONES EN LOS CU - EXPANDIDOS

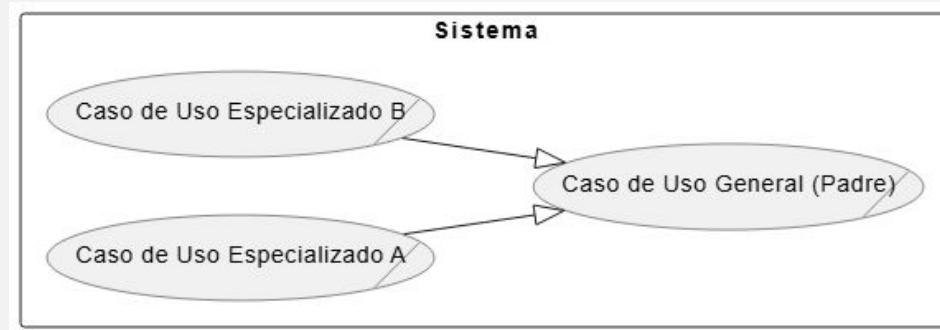
RELACIÓN 3: Generalización

¿Qué es?

- El caso de uso hijo hereda el comportamiento y propósito del padre, pero añade o redefine parte de ese comportamiento. El hijo puede especializarse sin que el padre se entere de los cambios.

¿Cuándo usarla?

- Esta relación ha caído en desuso por poder llegar a ser ambigua.
- Pero la seguimos usando para actores

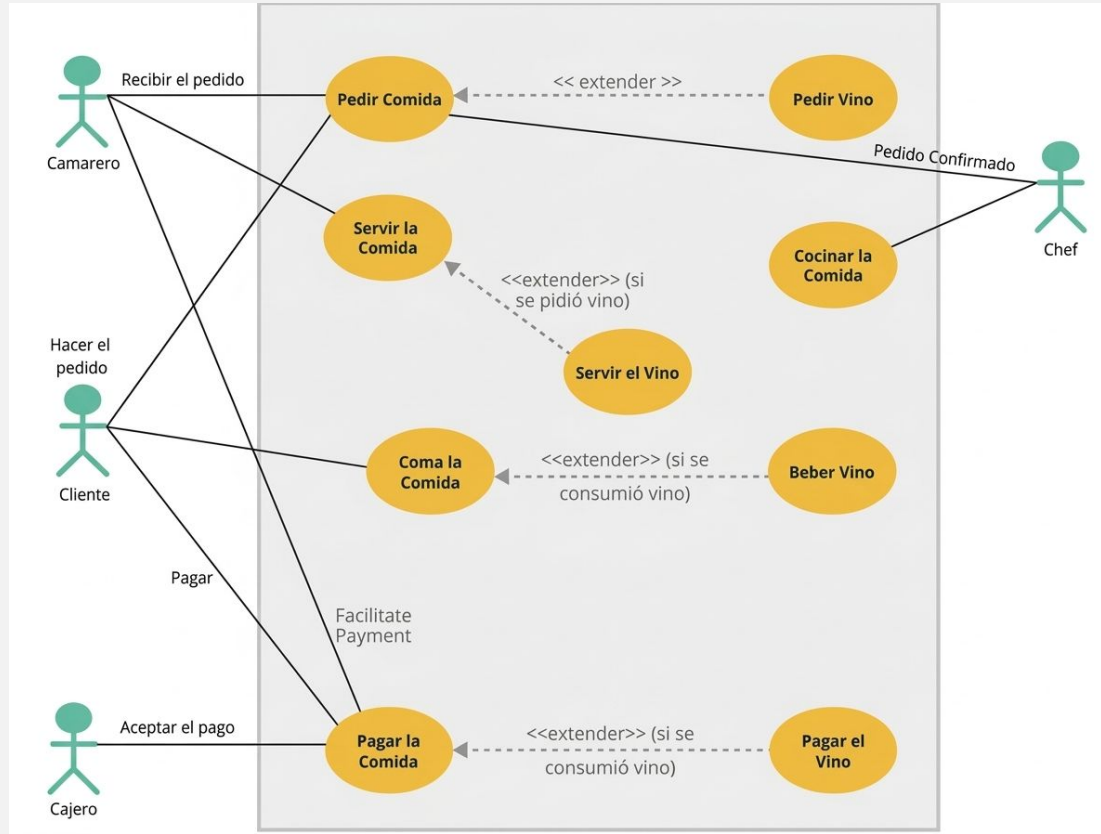


¿Include o Extend?

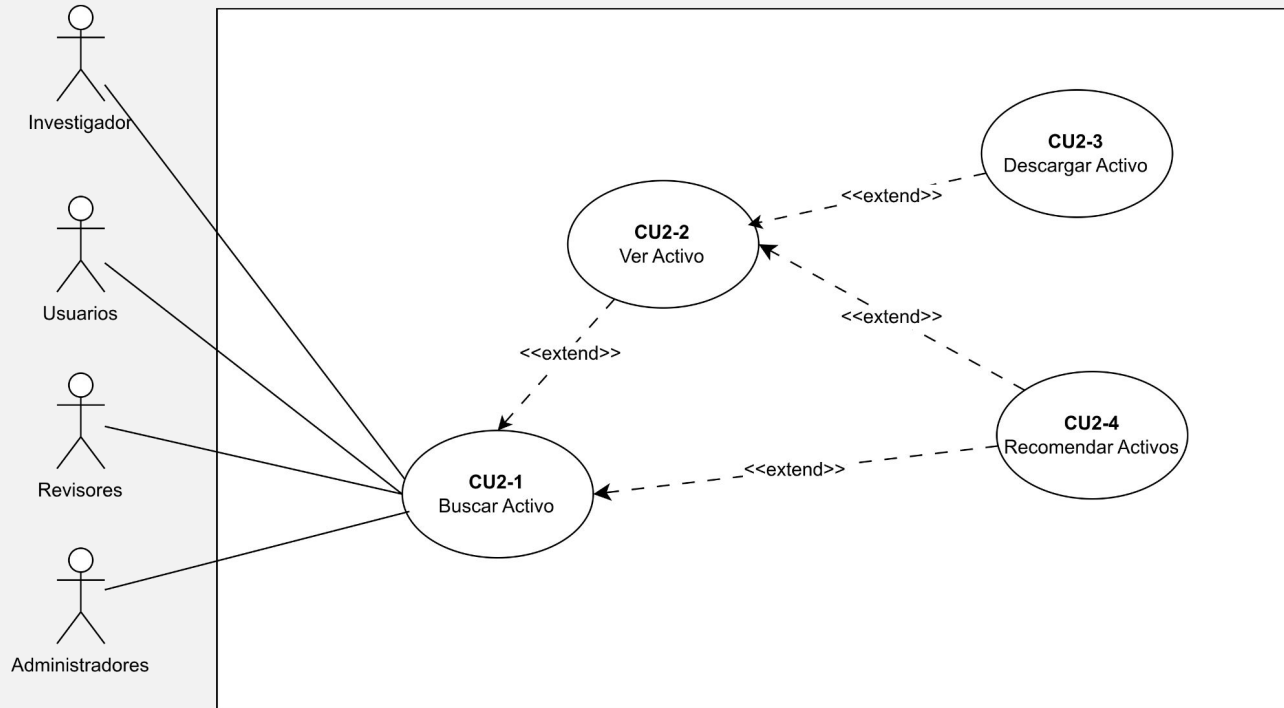
#	Caso de Uso A	Caso de Uso B	Pista	Respuestas
1	Procesar Pedido	Verificar Stock	B siempre ocurre cuando se procesa un pedido	¿?
2	Realizar Pago	Enviar Notificación	La notificación puede o no enviarse	¿?
3	Subir Documento	Validar Formato del Archivo	Siempre se valida al subir	¿?
4	Generar Reporte	Exportar a PDF	Solo se exporta si el usuario lo solicita	¿?

¿Include o Extend?

#	Caso de Uso A	Caso de Uso B	Pista	Respuestas
1	Procesar Pedido	Verificar Stock	B siempre ocurre cuando se procesa un pedido	include
2	Realizar Pago	Enviar Notificación	La notificación puede o no enviarse	extend
3	Subir Documento	Validar Formato del Archivo	Siempre se valida al subir	include
4	Generar Reporte	Exportar a PDF	Solo se exporta si el usuario lo solicita	extend



CU2 Búsqueda y Recomendaciones



DESCRIPCIONES TEXTUALES

Paso 4: Describir los casos de uso exandidos

Campo	Descripción
Nombre del caso de uso	Nombre representativo, usando verbo + sustantivo (ej: "Realizar Pedido")
Identificador	Código único (ej: CU1-1). Facilita la trazabilidad
Actores	Quién inicia y quiénes participan. Indicar cuál es el actor iniciador
Propósito	¿Para qué sirve este caso de uso? Una frase clara y concisa
Resumen	Descripción breve del flujo completo en 2-4 oraciones
Condiciones previas	¿Qué debe ocurrir antes de que este caso de uso pueda ejecutarse?
Secuencia normal	Pasos numerados del flujo feliz (cuando todo sale bien)
Excepciones / Cursos alternativos	¿Qué pasa si algo falla o hay variaciones? Referenciar el paso donde ocurre
Condiciones posteriores	¿Cómo queda el sistema después de ejecutarse?
Prioridad	Alta / Media / Baja
Comentarios	Notas adicionales relevantes para el equipo

DESCRIPCIONES TEXTUALES

Paso 4: Describir los casos de uso expandidos

Campo	Descripción
Nombre del caso de uso	de usoTomar Pedido
Identificador	CU-01
Actores	Camarero (iniciador), Cliente
Propósito	Registrar el pedido del cliente para su preparación en cocina
Resumen	El caso de uso inicia cuando el cliente está listo para ordenar. El camarero toma la orden, la registra en el sistema y la envía a cocina. Finaliza cuando cocina confirma recepción del pedido.
Condiciones previas	El cliente debe estar asignado a una mesa. El camarero debe estar autenticado en el sistema.
Secuencia normal	1. El cliente indica al camarero que está listo para ordenar. 2. El camarero abre el pedido en el sistema. 3. El cliente selecciona los productos del menú. 4. El camarero registra los productos y cantidades. 5. Se invoca "Registrar Operación". 6. El sistema envía el pedido a cocina. 7. Cocina confirma recepción. 8. El sistema muestra confirmación al camarero.

DESCRIPCIONES TEXTUALES

Paso 4: Describir los casos de uso expandidos

Excepciones / Cursos alternativos	Paso 3: Si un producto no está disponible, el sistema lo indica y el camarero sugiere alternativas. Paso 6: Si hay error de comunicación con cocina, el sistema guarda el pedido y reintenta.
Condiciones posteriores	posterioresEl pedido queda registrado en el sistema con estado "En preparación" y asociado a la mesa del cliente.
Prioridad	Alta
Comentarios	Si el cliente solicita vino, se extiende al caso de uso "Servir Vino" (CU-02).



Modelado y Construcción Sistemas

Introducción a UML · Diagramas de Clases y Secuencia ·
Herramientas



¿QUÉ ES UML?

- UML = Unified Modeling Language (Lenguaje Unificado de Modelado).
- Creado en los 90 por Booch, Rumbaugh y Jacobson (los "Tres Amigos")
- Estándar de la OMG (Object Management Group) desde 1997
- No es un lenguaje de programación: es un lenguaje visual para diseñar sistemas
- Permite comunicar el diseño a desarrolladores, arquitectos y stakeholders técnicos
- UML 2.x define 14 tipos de diagramas, divididos en estructurales y de comportamiento

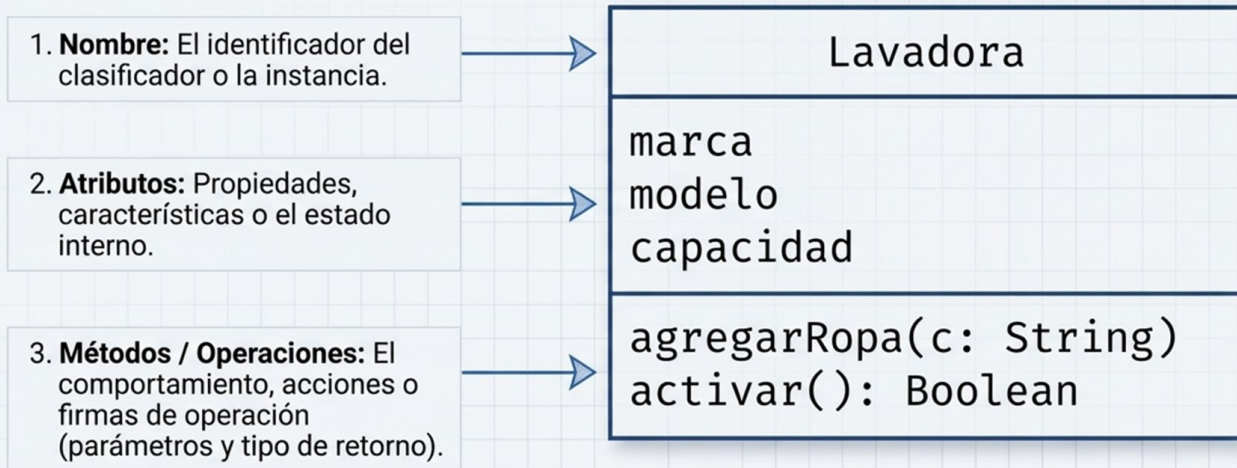
TIPOS DE DIAGRAMAS UML

Diagramas Estructurales (el "qué es")	Diagramas de Comportamiento (el "qué hace")
Diagrama de Clases	Diagrama de Secuencia
Diagrama de Objetos	Diagrama de Casos de Uso
Diagrama de Componentes	Diagrama de Actividades
Diagrama de Despliegue	Diagrama de Estados
Diagrama de Paquetes	Diagrama de Comunicación
Diagrama de Perfil	Diagrama de Interacción
Diagrama de Estructura Compuesta	Diagrama de Tiempos

Diagrama de Clases: la columna vertebral del diseño

- Un diagrama de clases modela la estructura estática del sistema: qué entidades existen, qué datos tienen y cómo se relacionan entre sí.
- **¿Qué representa una clase?:** Una descripción de un grupo de objetos, todos con roles similares en el sistema, que consta de:
 - Las características estructurales (atributos) definen lo que los objetos de la clase "saben"
 - Representar el estado de un objeto de la clase.
 - Son descripciones de las características estructurales o estáticas de una clase.
 - Las características de comportamiento (operaciones) definen lo que los objetos de la clase "pueden hacer"
 - Define la forma en que los objetos pueden interactuar
 - Las operaciones son descripciones de características dinámicas o de comportamiento de una clase.

La Anatomía de una Clase

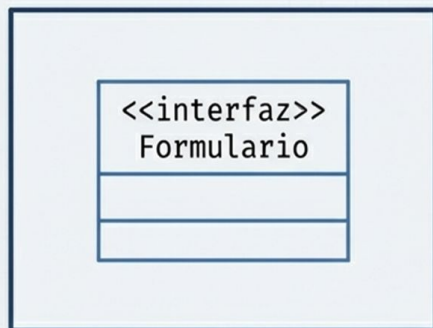


Control de Visibilidad y Encapsulamiento

Los modificadores de acceso definen qué componentes externos pueden interactuar con los datos.

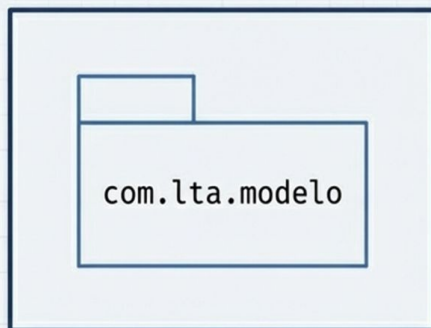
Símbolo	Modificador	Icono	Alcance
+	Público (Public)		Accesible desde cualquier otra clase u objeto.
-	Privado (Private)		Accesible únicamente dentro de la misma clase. (Requiere getters/setters).
#	Protegido (Protected)		Accesible por la propia clase, sus subclases (herencia) y el mismo paquete.
~	Paquete (Package)		Accesible solo por clases dentro del mismo paquete.

Ampliando el Vocabulario Estructural



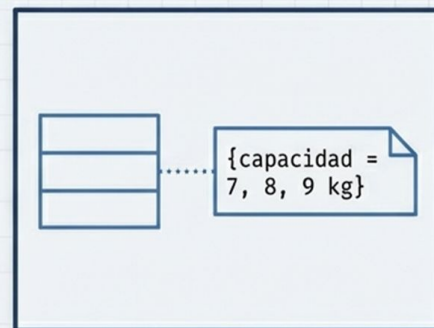
Estereotipos

Extienden y personalizan un elemento de UML. Actúan como etiquetas semánticas.



Paquetes

Organizan elementos del modelo en grupos lógicos y coherentes.

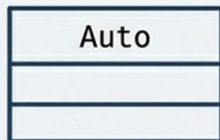


Notas y Restricciones

Comentarios adjuntos o reglas de negocio (restricciones) formales que limitan el comportamiento.

Conectando el Sistema: Asociaciones

Una asociación es una relación básica que conecta dos o más clases estructuralmente.



Viaje

Asociación Bidireccional:

Ambas clases son conscientes de la otra. (Relación por defecto).



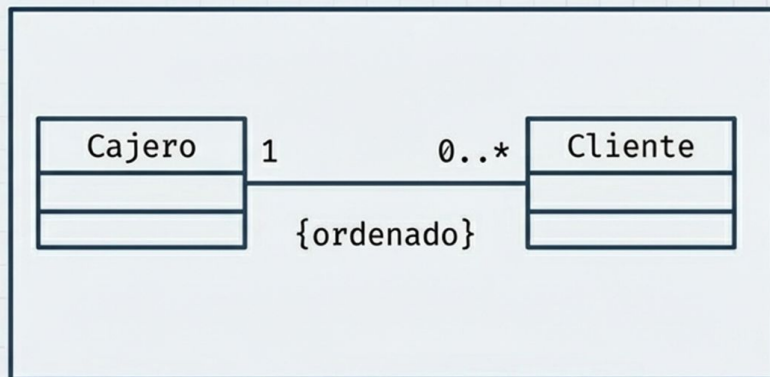
Televisión

Asociación Unidireccional:

El Control Remoto conoce a la Televisión (envío de mensajes), pero la TV no interactúa de vuelta.

Multiplicidad y Reglas de Negocio

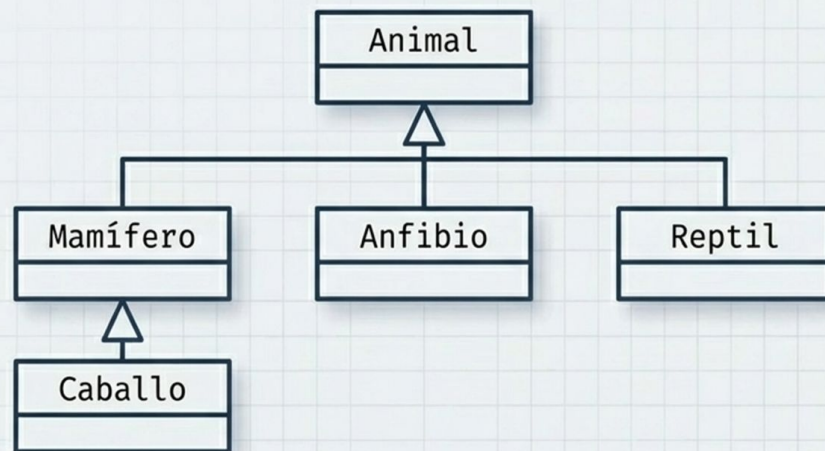
1	Uno a uno exacto
0..1	Cero a uno
1..*	Uno a muchos
0..*	Cero a muchos



Un cajero atiende a cero o muchos clientes, pero debe seguir una restricción estructural: la fila de clientes se atiende de forma ordenada.

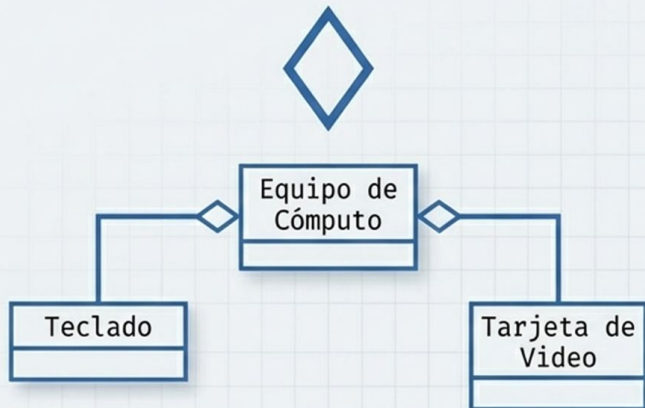
Herencia y Generalización

El pilar de la reusabilidad orientada a objetos.



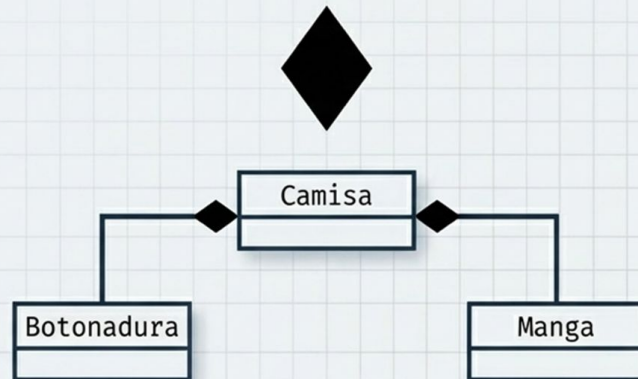
La **Generalización** es el proceso abstracto de identificar características comunes. La **Herencia** es el mecanismo que transfiere esos atributos y métodos hacia las subclases.

El Todo y las Partes: Agregación vs. Composición



Agregación (Relación Débil)

Los componentes pueden existir independientemente del todo. Si el equipo se destruye, el teclado sigue existiendo física y lógicamente.



Composición (Relación Fuerte)

Dependencia total. Los componentes no pueden existir sin el todo. Si la camisa deja de existir, sus partes contextuales desaparecen del sistema.

Relaciones Transitorias y Contratos

Dependencia (Uso momentáneo)



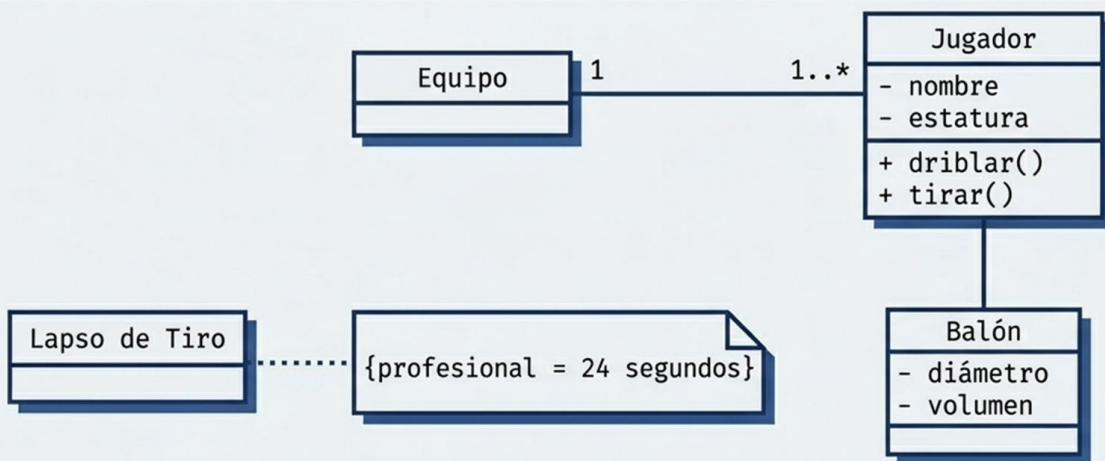
Una clase utiliza a otra para cumplir una tarea específica. Un cambio en la clase dependida impactará a la dependiente.

Realización (Implementación de Interfaces)



Un contrato estricto. La clase se compromete a implementar las operaciones definidas (ej. escribir, borrar) por la interfaz.

Poniendo Todo en Práctica: Simulador Deportivo



Un diagrama **de contexto** simplificado demuestra cómo interactúan los objetos **dinámicamente** manteniendo **reglas estructurales estrictas**.

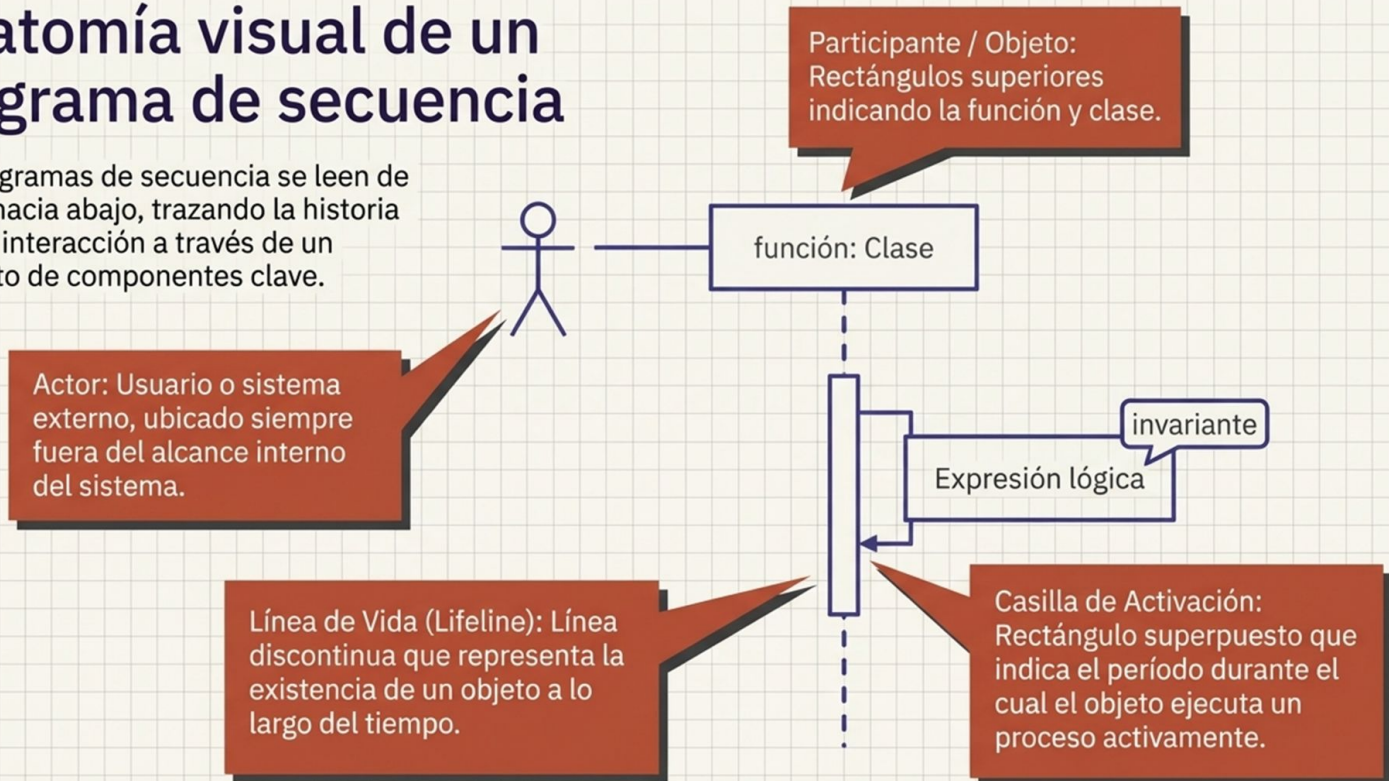
Diagrama de Secuencia: el sistema en movimiento

Un diagrama de secuencia modela cómo interactúan los objetos a lo largo del tiempo para llevar a cabo un caso de uso específico.

- Propósito del diagrama de secuencia
- Modelar la interacción de alto nivel entre objetos activos en un sistema
- Modelar la interacción entre instancias de objetos dentro de una colaboración que realiza un caso de uso
- Modelar la interacción entre objetos dentro de una colaboración que realiza una operación
- Modele interacciones genéricas (mostrando todas las rutas posibles a través de la interacción) o instancias específicas de una interacción (mostrando solo una ruta a través de la interacción)

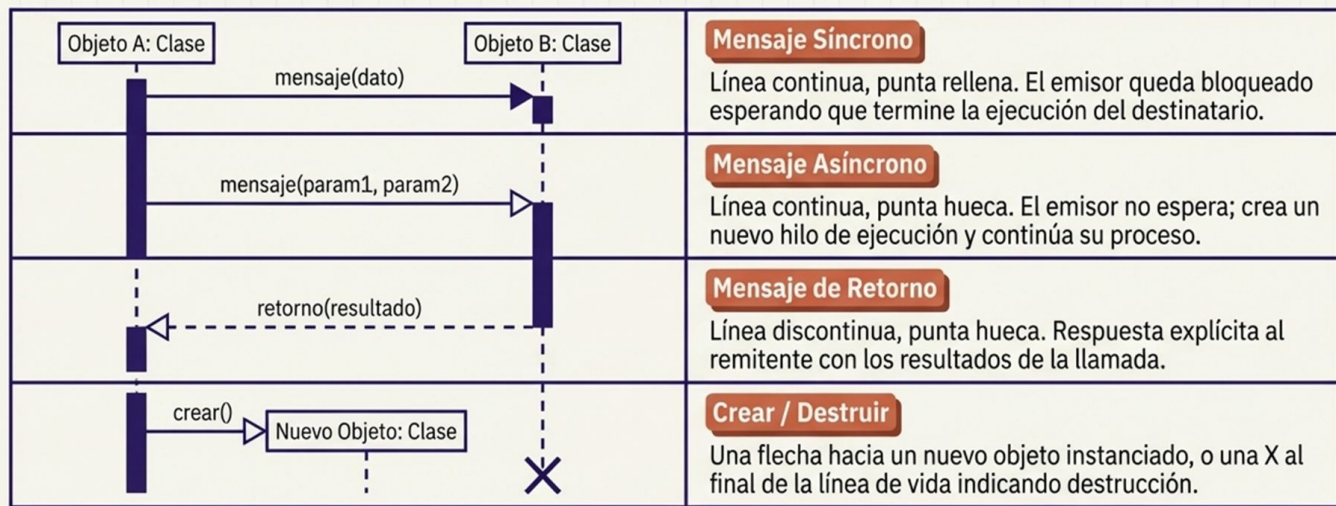
Anatomía visual de un diagrama de secuencia

Los diagramas de secuencia se leen de arriba hacia abajo, trazando la historia de una interacción a través de un conjunto de componentes clave.



El lenguaje exacto de los mensajes

La forma de la flecha dictamina el comportamiento del sistema. Los objetos se vuelven activos al recibir un mensaje, lo que equivale a la llamada de un método en programación.

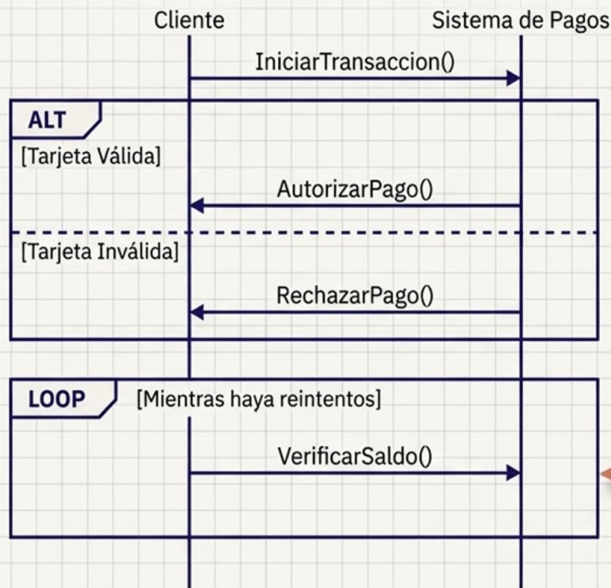


Controlando la complejidad con fragmentos combinados

Las interacciones de software rara vez son lineales. Los 'marcos' o fragmentos combinados (Packages/Frames en UML 2.0) permiten modelar la lógica de negocio condicional directamente en el diagrama.

El Marco ALT (Alternativas):

Representa decisiones mutuamente excluyentes (if/else). Evalúa condiciones de guarda.

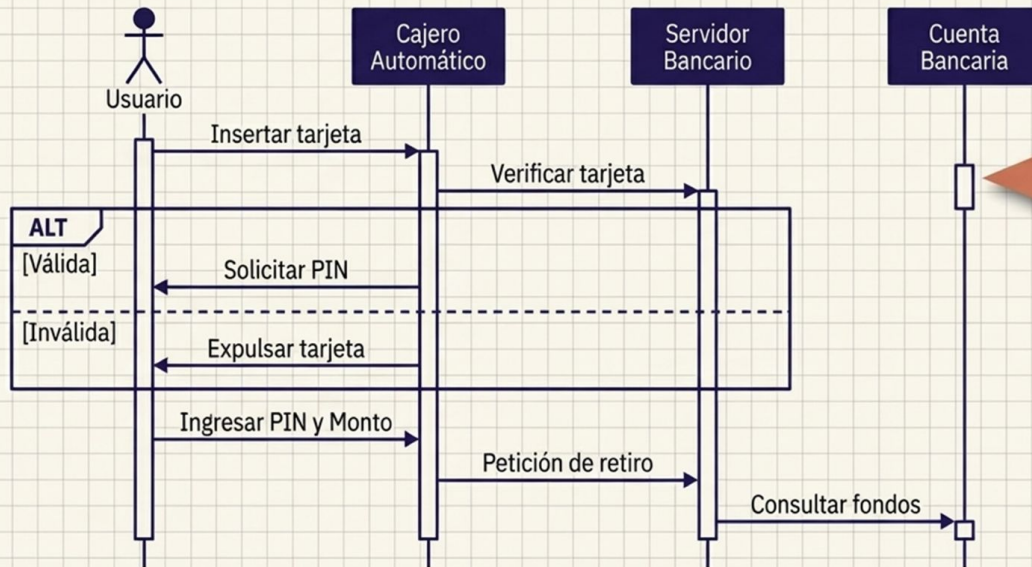


El Marco LOOP (Iteración):

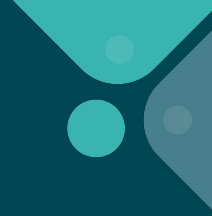
Representa acciones repetitivas o reintentos (sentencias DO/FOR). Modela procesos que se ejecutan múltiples veces.

Caso de estudio: La dinámica de un cajero automático

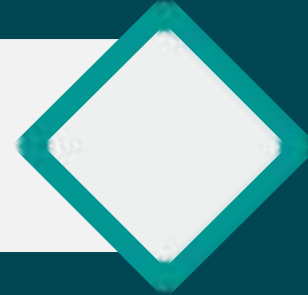
Reconstrucción de un flujo de retiro de efectivo ilustrando la colaboración entre entidades externas y sistemas internos.



Nota Visual: Las casillas de activación de la Cuenta Bancaria son mínimas, demostrando su inactividad durante la validación inicial del hardware.



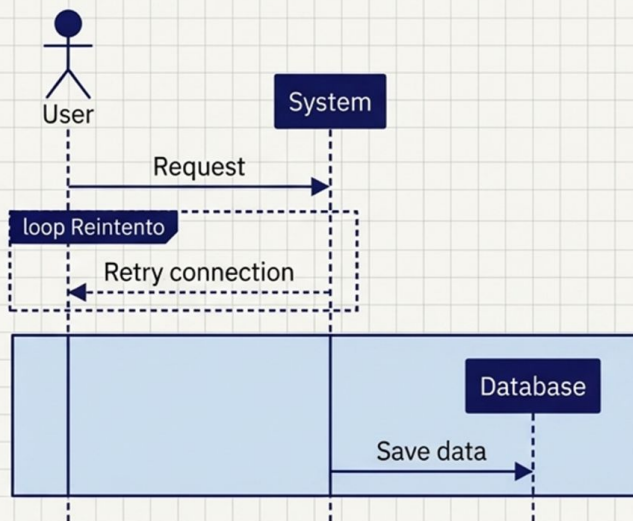
Herramientas de modelado



Modelado ágil utilizando la sintaxis de Mermaid

Mermaid democratiza UML permitiendo dibujar diagramas de secuencia precisos utilizando únicamente texto Markdown. Ideal para integración en repositorios de código y control de versiones.

```
sequenceDiagram
    actor User
    participant System
    User->>System: Request
    %% Comentario de validación
    loop Reintento
        System-->>User: Retry connection
    end
    rect rgb(200, 220, 240)
        System->>Database: Save data
    end
```



El ecosistema Enterprise para diseño UML

Para arquitecturas complejas, las herramientas especializadas ofrecen control absoluto y trazabilidad. Una comparativa de los líderes del mercado.



Visual Paradigm

Líder absoluto. Suite completa, interfaz intuitiva e integración robusta con IDEs como Eclipse. Soluciones Cloud y locales.



Lucidchart

Enfoque web superior. Excelentes características de colaboración en tiempo real, ideal para equipos distribuidos y ágiles.



Enterprise Architect

Extremadamente robusta y compleja. Soporta lenguajes de modelado extensos diseñados para la escala de grandes organizaciones.



StarUML & ArgoUML

Alternativas de código abierto. StarUML es altamente personalizable; ArgoUML destaca como estándar excelente para fines educativos.

Alternativas gratuitas para arranques rápidos

Si el objetivo es la velocidad y no se requiere una suite Enterprise pesada, el ecosistema de código abierto y freemium ofrece soluciones potentes.

Draw.io (EdrawMax online)

- Flexible y centrado en la privacidad
- Soporta múltiples tipos de topología de red y estándares UML

Gaphor

- Potente pero fácil de usar
- Soporta múltiples lenguajes de modelado: UML, SysML, RAAML y C4

UMLet

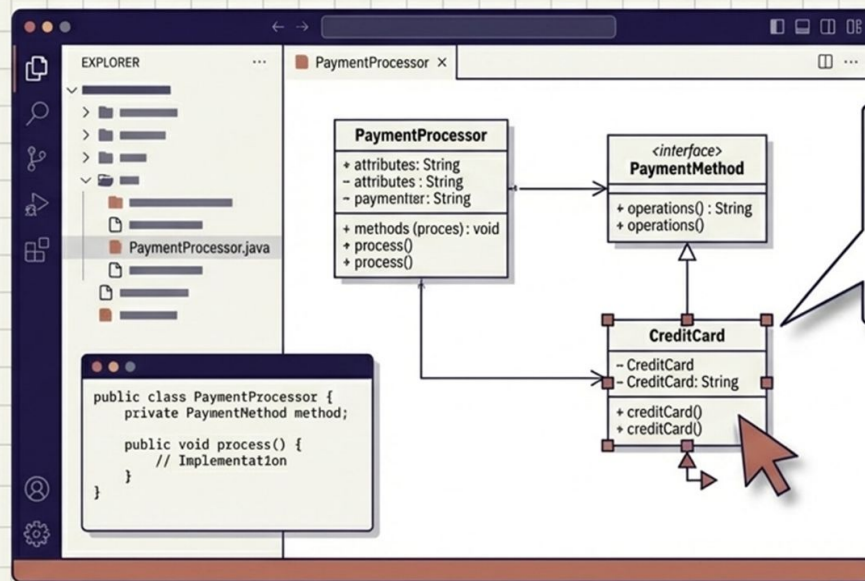
- Interfaz altamente minimalista
- Diseñado para diagramación extremadamente rápida
- Exportación directa a PDF/EPS

Umbrello

- Generador de código integrado
- Soporta el estándar industrial XMI para portabilidad

Diagramación nativa sin abandonar el editor de código

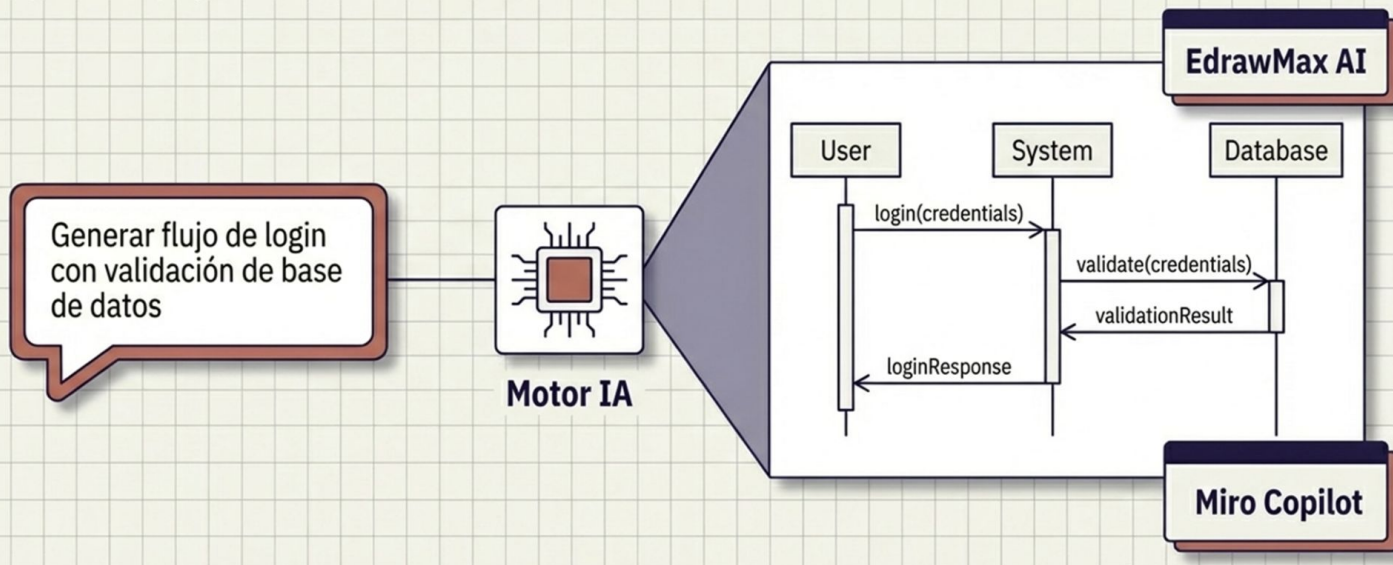
El modelado es más efectivo cuando vive junto al código. Extensiones como bigUML Modeling Tool integran UML directamente en Visual Studio Code, permitiendo edición gráfica y textual híbrida sin pérdida de contexto.



Requiere Java JRE 17+.
Utiliza tecnología GLSP
y ModelServer para
sincronización en
tiempo real.

La inteligencia artificial como asistente de diagramación

La evolución de herramientas como EdrawMax y Miro AI permite la creación de diagramas de secuencia a partir de descripciones en lenguaje natural. Esto reduce la curva de aprendizaje y acelera el diseño arquitectónico.

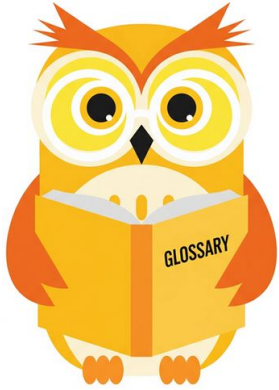


REFERENCIAS

- **Object Management Group (OMG).** (2017). *Unified Modeling Language (UML) Version 2.5.1*. <https://www.omg.org/spec/UML/2.5.1/>
- **Fowler, M.** (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.
- **Jacobson, I., Christerson, M., Jonsson, P., & Övergaard, G.** (1992). *Object-Oriented Software Engineering: A Use Case Driven Approach*. Addison-Wesley.
- **Larman, C.** (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* (3rd ed.). Prentice Hall.
- **Cockburn, A.** (2000). *Writing Effective Use Cases*. Addison-Wesley.



Ejemplo práctico



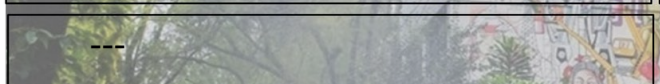
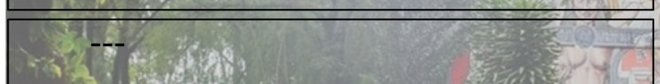
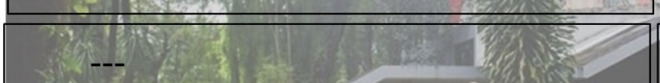
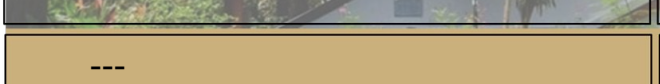
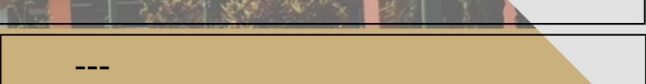
Practiquemos lo aprendido



Nombre de la actividad:	---
Cantidad de participantes:	---
Doy fe que esta actividad está planificada en dtt (Sí/No):	---

Hora de inicio:	---
Hora de fin:	---
Duración (min):	---

Participantes: llenar las siguientes cajas de texto (tomar información del chat del meet)

Recursos

- Link del Quizz (cada instructor colocara su link)



DUDAS ¿?



**¡GRACIAS POR SU
ATENCIÓN!**

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA
QUE TE SURJA EN LA SEMANA